

The background of the slide is a photograph of two men in business suits. The man on the left is wearing glasses and a striped tie, looking down at a tablet. The man on the right is smiling and pointing at the tablet with a pen. The scene is brightly lit, suggesting an office environment.

Forsta Open Dialer API. Inbound and IVR Support. v.1.2

Contents

Contents.....	2
Introduction	4
Inbound calls support	4
Agent Management	4
Case 1. Pure inbound survey.....	4
Case 2. Inbound surveys without logged in users.....	5
Case 3. Blended surveys.....	5
Playing messages	5
IVR support	6
Cases	6
Case 1. Outbound. Predictive mode. Start interview by live agent, transfer to IVR then.	7
Case 2. Outbound. Automatic mode. Start interview by live agent, transfer to IVR then.	8
Case 3. Outbound. Automatic mode. Start interview by IVR, transfer to a live agent then.....	9
Case 4. Outbound. Automatic mode. IVR only.	10
Case 5. Inbound. IVR only.	11
Case 6. Inbound. Start interview by IVR, transfer to a live agent then.....	12
Case 7. Inbound. Start interview by live agent, transfer to IVR then.	13
Open Dialer Inbound and IVR Support API	14
[Dialer->CATI] NotifyInboundCall(inboundCallId, inboundLineId, callerPhoneNumber)	14
[CATI->Dialer] ConnectInboundCall(predictiveCampaignId, inboundCampaignId, inboundCallId)	14
[CATI->Dialer] ConnectInboundCallToAgent(inboundCampaignId, agentId, inboundCallId).....	14
[CATI->Dialer] DropInboundCall(inboundCallId).....	14
[CATI->Dialer] TransferToIvrAgent()	14
[Dialer->CATI] NotifyIvrAgentTransfer()	14
[CATI->Dialer] IvrRenderVoiceXml().....	14
[Dialer->CATI] NotifyIvrSubmit()	14
[CATI->Dialer] TransferToLiveAgent()	14
[Dialer->CATI] NotifyLiveAgentTransfer()	14
Virtual IVR Agent Login	15
Questions	16
VoiceXML Subset.....	17
Limitations.....	17
Play a Synthesized Voice.....	17

Play an Audio File.....	17
User Interaction with DTMF.....	17
Built-in Grammars.....	18
Wrong Input Handling.....	19
Sending Data to CATI	20
Audio Recording.....	20
Generic DTMF Recognizer Properties	21
timeout.....	21
termchar.....	22
termmaxdigits	22
Interdigittimeout.....	22

Introduction

Inbound calls support will allow users to utilize their call centers spare capacity. A lot of CATI call centers are less busy in the morning unless they are doing B2B calls so they can perform inbound type of jobs in the morning hours.

We also need this functionality to support IVR because the majority IVR scenarios are inbound.

One of the most important purpose of the IVR support is to use Agent's resources more efficiently.

Inbound calls support

Agent Management

There are 2 agent types:

1. Live agent
2. IVR Virtual agent

There are 4 main modes in relation to survey:

1. Agent is logged in to an **inbound** survey and he/she handles inbound calls for the survey only.
2. Agent is logged in, but **without a survey** specified.
3. Agent is logged in to an **outbound** (predictive or non-predictive) survey. The agent can be borrowed temporary from the survey to handle an inbound call.
4. Agent is logged in to a **blended** (outbound+inbound) survey and he/she handles calls (of the both types) for the survey only.

For non-predictive modes CATI can manage agent resources and instruct dialer to connect an inbound call to particular available agent. Ideally all (not inbound only) agents should be able to log in without a survey specified and thus should be able to pass interviews from any non-predictive survey.

For predictive mode agents are controlled by dialer. So CATI do not specify an agent explicitly and dialer chooses an available agent by itself. CompleteCall() or GoReady() returns the agent back to the pool of agents ready to handle a predictive call.

Q: *do we need to support routing an inbound call to a specific agent logged in to a predictive survey?*

Case 1. Pure inbound survey

- Dialer receives an inbound call and sends notification to CATI
- Based on the information supplied by dialer (inbound line id, caller phone number) CATI identifies a required survey and locates a record for the inbound call (or takes a fresh record... or creates a new record)
- CATI selects a first available agent from a list of available agents and instructs the dialer to route the inbound call (specified by id received in the notification from dialer) to the agent by sending **ConnectInboundCall** request.
- If CATI cannot identify a survey or a call or it cannot locate an agent then it needs to send a message to the dialer that the call can't be handled. Probably "play" message should be send to dialer.
DropInboundCall(inboundCallId) method can be used for dropping not routed call if it can't be routed for some reason.
- How we should deal with the case if all people are busy? Send a message to a dialer to play a specific message, put message in the queue and when person is available assign call to the user.

Case 2. Inbound surveys without logged in users

- Dialer receives an inbound call and sends notification to CATI
- Based on the information supplied by dialer (inbound line id, caller phone number) CATI identifies a required survey and locates a record for the inbound call (or takes a fresh record... or creates a new record)
- If CATI cannot identify a survey or a call then it needs to send a message to the dialer that the call can't be handled.
- When CATI finds a call it does:
 - For predictive survey CATI sends **ConnectInboundCall(new survey, call)** to the dialer. Dialer should select an appropriate agent, 'switch' campaign, route the inbound call (specified by id received in the notification from dialer) to the agent and then 'switch' campaign back after the call is completed.
 - For non-predictive survey agents should be able to work on multiple surveys. High priority call will be placed for specific survey. To connect the inbound call to a specific agent **ConnectInboundCallToAgent(survey, agent, call)** request is used. This means dialer should support mode of logging in without a specific survey.

Case 3. Blended surveys

- Dialer receives an inbound call and sends notification to CATI
- Based on the information supplied by dialer (inbound line id, caller phone number) CATI identifies a required survey and locates a record for the inbound call (or takes a fresh record... or creates a new record)
- If CATI cannot identify a survey or a call then it needs to send a message to the dialer that the call can't be handled.
- When CATI finds a call it does:
 - For predictive survey CATI sends it to dialer using **ConnectInboundCall(new survey, call)** command. This call should be handled by dialer as a priority call and should be assigned to an agent asap. Dialer is responsible to select an available agent.
 - For non-predictive survey the call is marked as "inbound" and is placed in a special queue. When survey is marked as "Allow inbound" CATI first look at this special queue when it tries to deliver call to an agent. When CATI locates an agent for a call it uses **ConnectInboundCallToAgent(survey, agent, call)** to connect the inbound call to the agent.

Playing messages

There are some cases when we probably need to play a message to respondent

- Unconditional introduction before connect to an agent
- After some timeout if inbound call is still not routed to an agent for some reason
- There is no agent available to handle the call

There are some questions

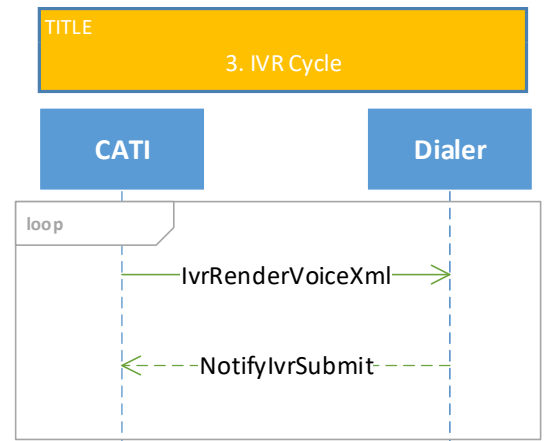
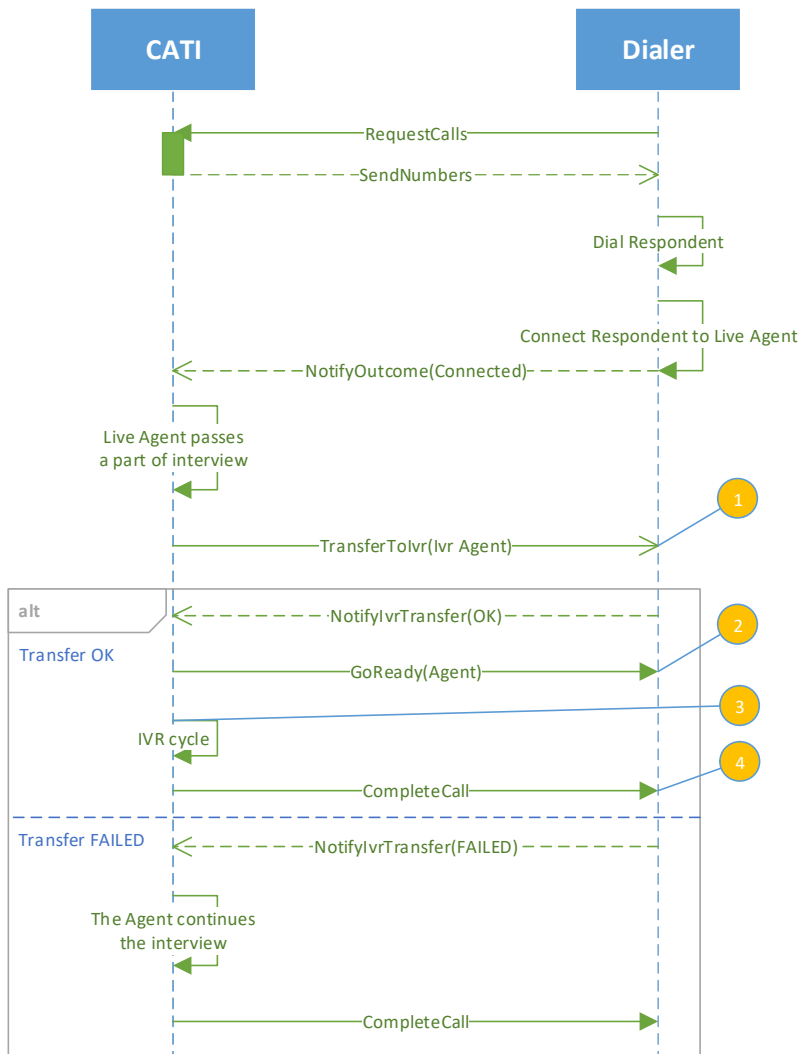
- Should the message be the same for all cases or we need a separate message for each case?
- How to setup the message(s)? As a campaign options?
- Should the message (about a delay or no agent available) been interrupted when the call is connected to an agent?

IVR support

Cases

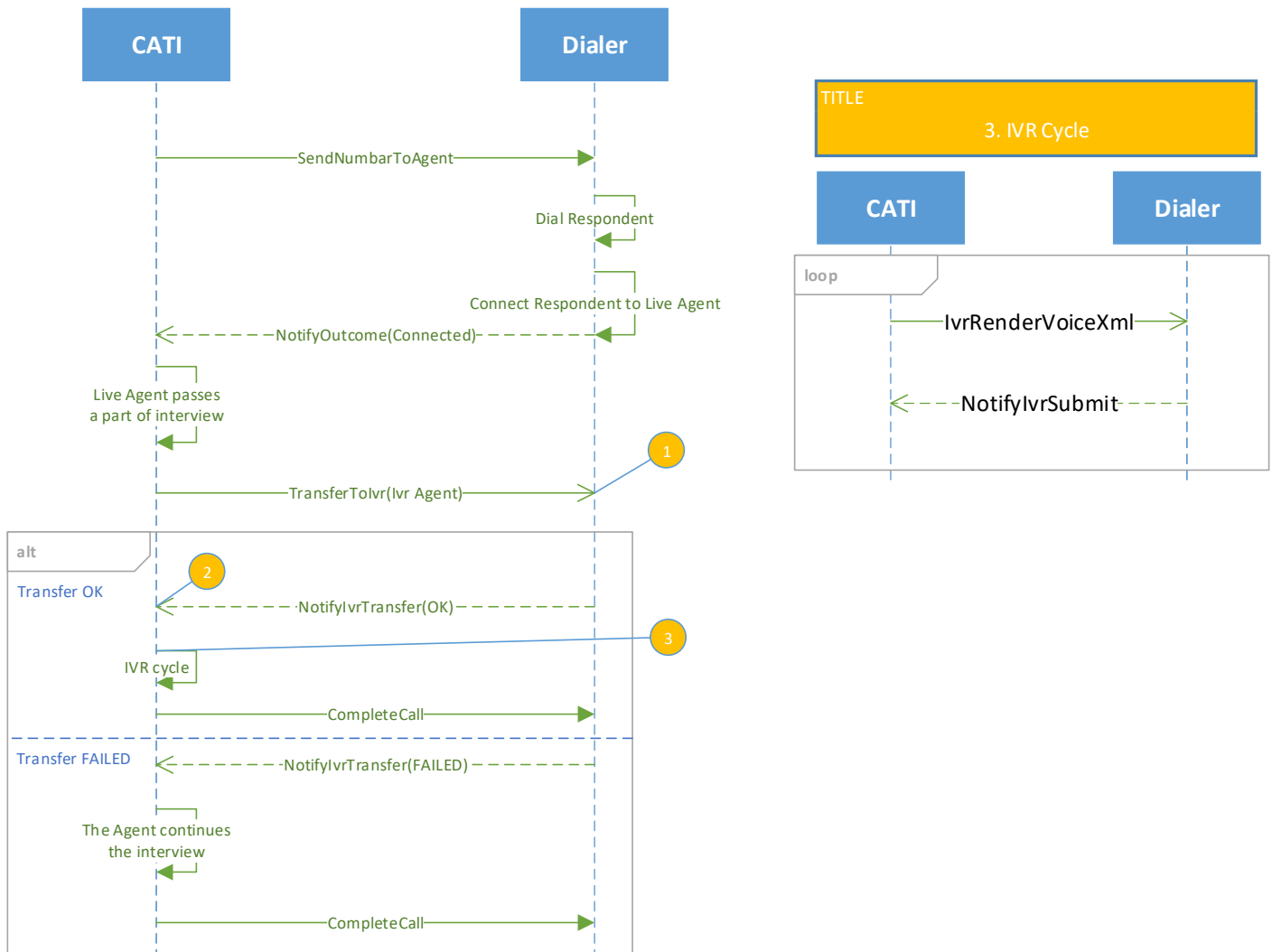
1. Outbound. Predictive mode. Start interview by live agent, transfer to IVR then.
2. Outbound. Automatic mode. Start interview by live agent, transfer to IVR then.
3. Outbound. Automatic mode. Start interview by IVR, transfer to a live agent then.
4. Outbound. Automatic mode. IVR only.
5. Inbound. IVR only.
6. Inbound. Start interview by IVR, transfer to a live agent then.
7. Inbound. Start interview by live agent, transfer to IVR then.
8. ? Predictive mode. Start interview by IVR, transfer to live agent then. – We don't plan to support this.
9. ? Predictive mode. IVR only. – We don't plan to support this.

Case 1. Outbound. Predictive mode. Start interview by live agent, transfer to IVR then.



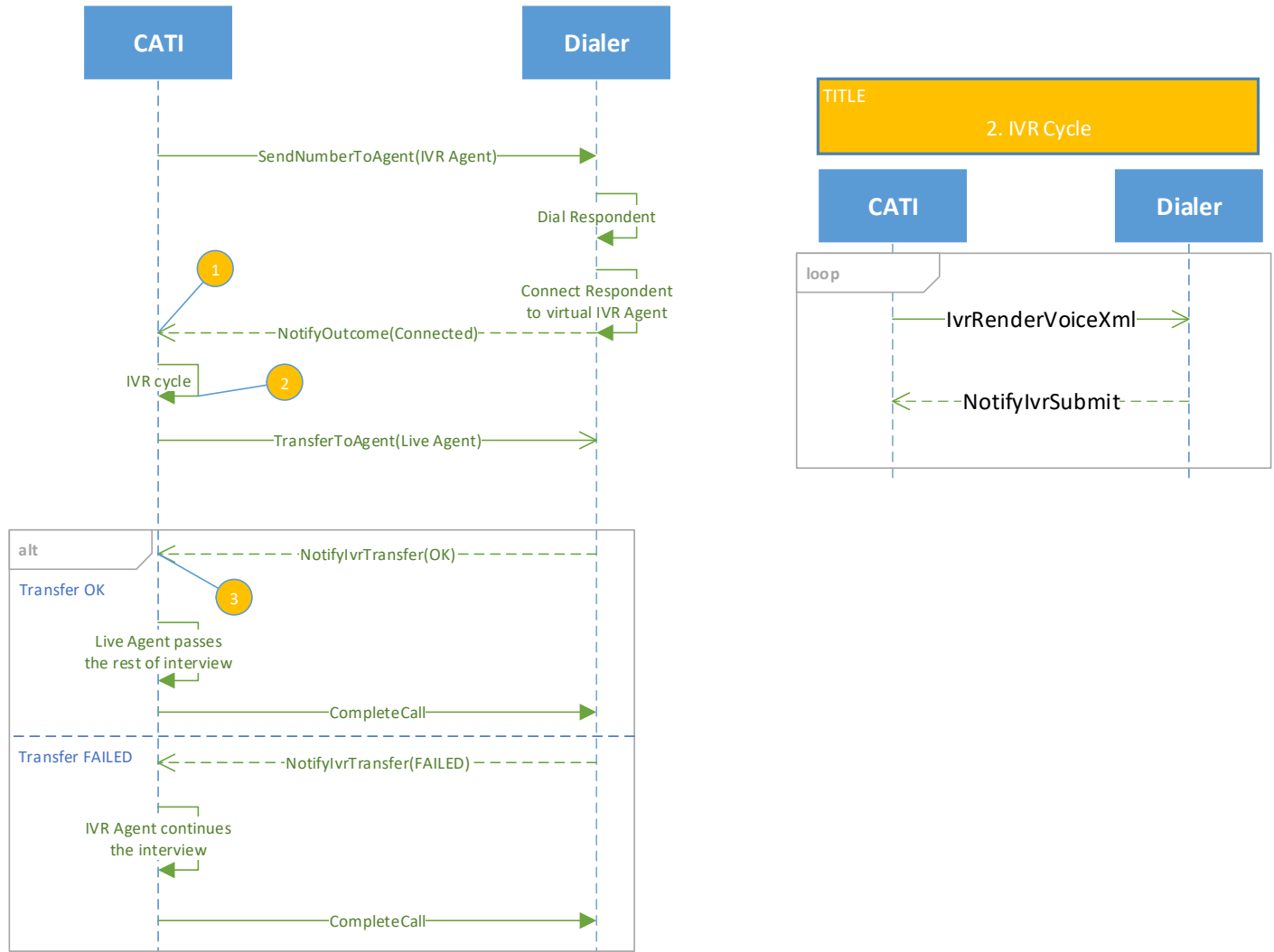
1. Who should manage the virtual IVR Agents – CATI or Dialer?
2. Should CATI send explicitly **GoReady** command after successful **TransferToIvr**?
4. Transferred to IVR call ends in the same way as usual with a **CompleteCall** request.

Case 2. Outbound. Automatic mode. Start interview by live agent, transfer to IVR then.



1. Who should manage the virtual IVR Agents – CATI or Dialer? Seems, doing that in CATI gives more flexibility.
2. The live Agent is ready for the next interview right after successful transfer to IVR.

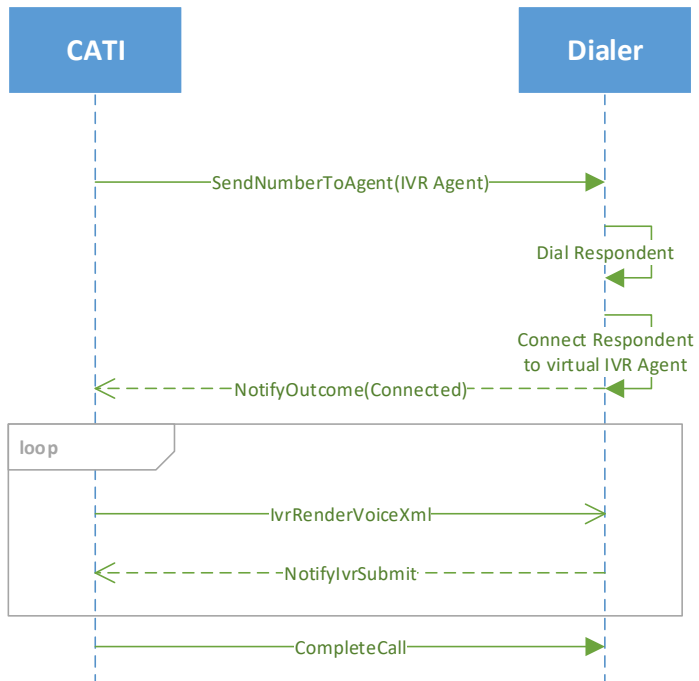
Case 3. Outbound. Automatic mode. Start interview by IVR, transfer to a live agent then.



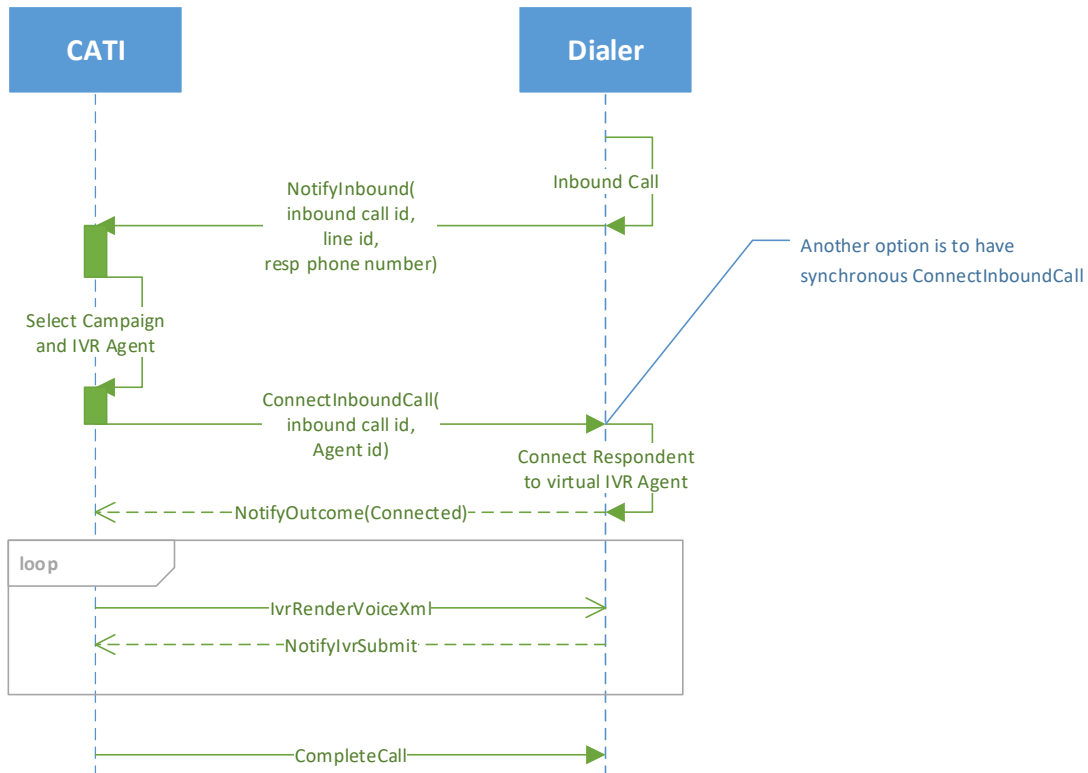
1. CATI should 'know' somehow that the Agent is a virtual IVR agent in order to start the IVR cycle.
3. The IVR Agent is ready for the next interview right after successful transfer to Live Agent

Transfer to agent can be done conditionally (for instance a special button is pressed by respondent) or unconditionally depending on the survey script logic. CATI (or dialer) selects a free agent. Also it should be handled somehow the case when no live agents available to handle the call.

Case 4. Outbound. Automatic mode. IVR only.

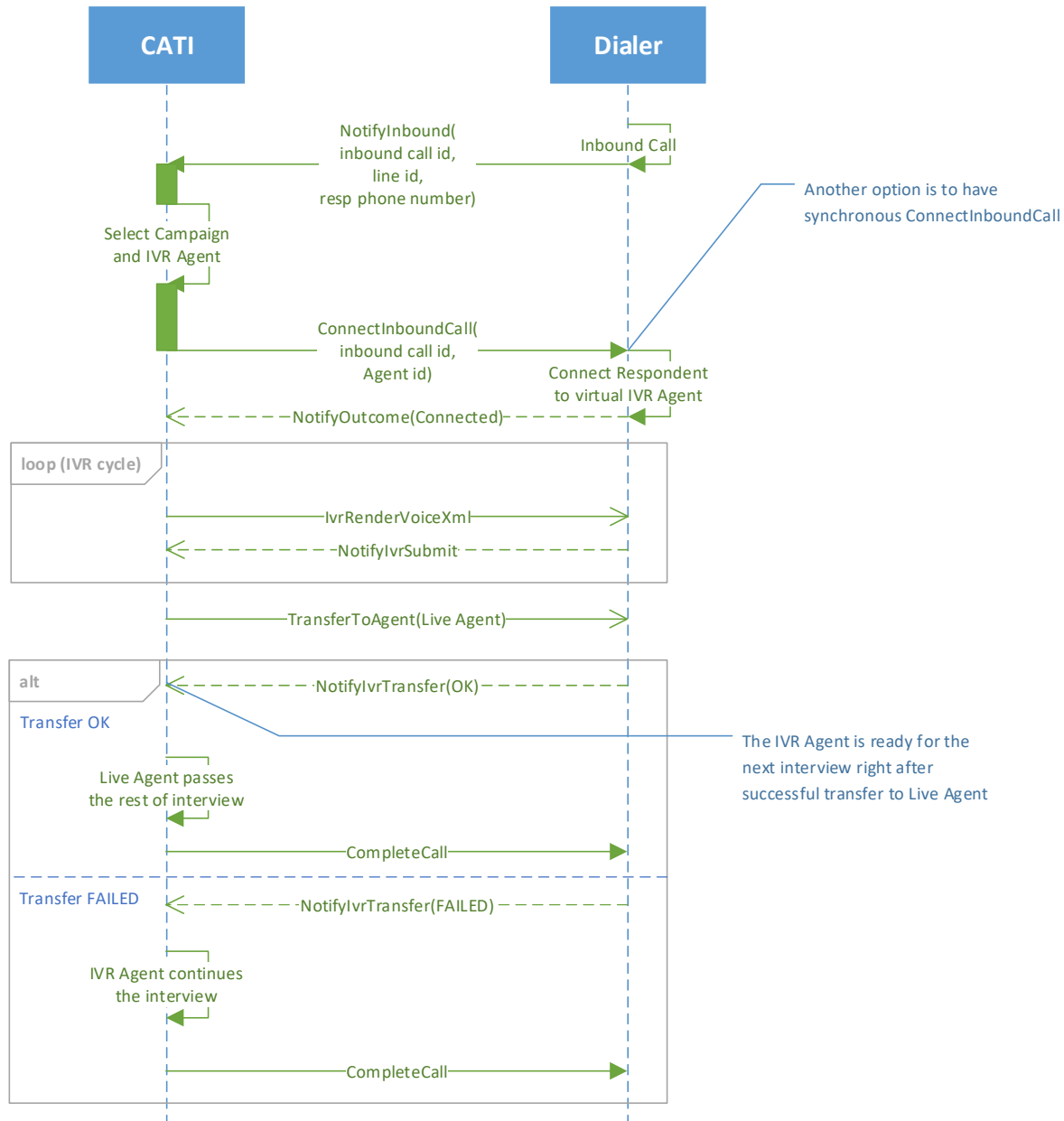


Case 5. Inbound. IVR only.



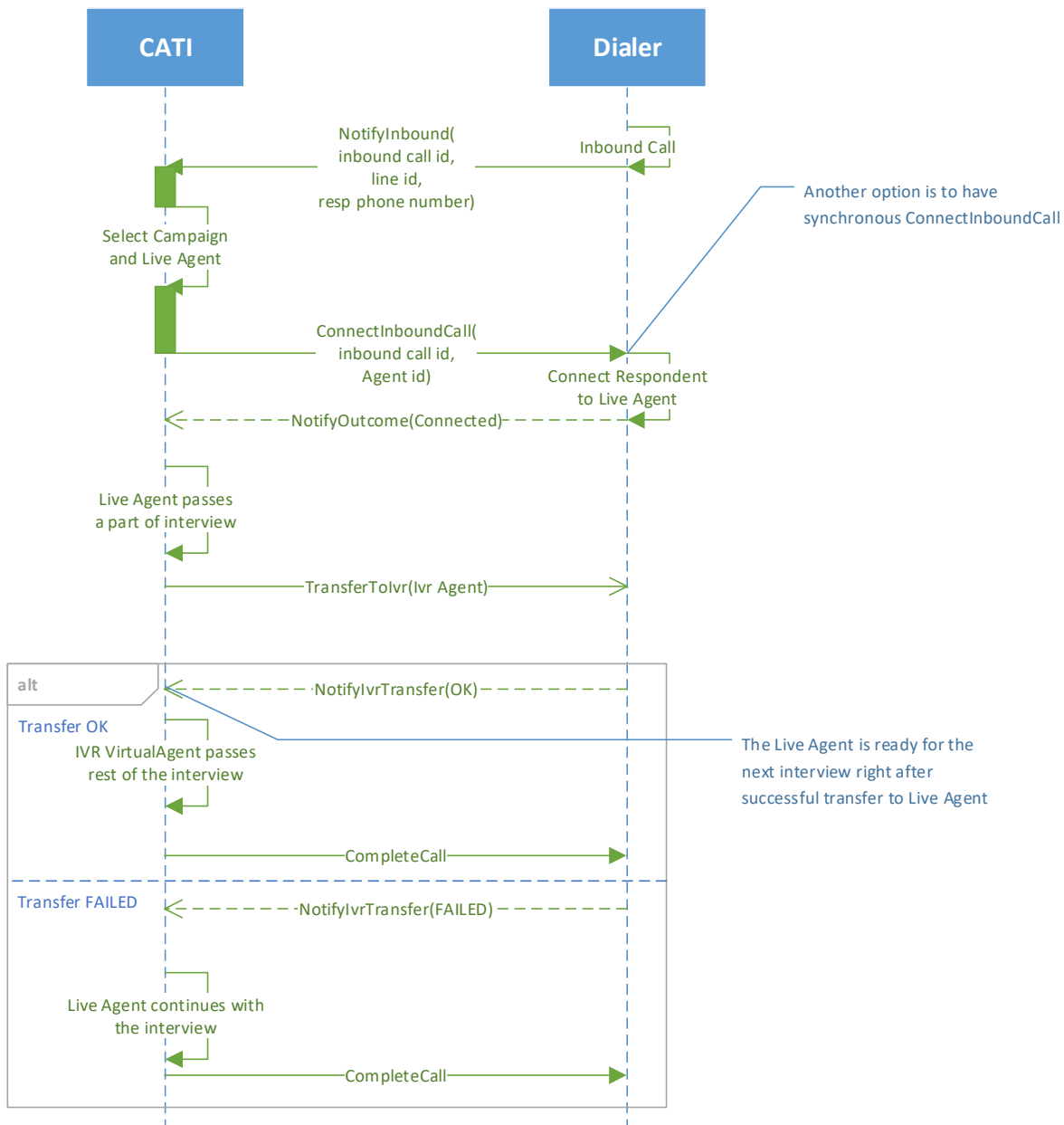
Note that the 'inbound call id' here is not CATI call id. It's a dialer side id that allows dialer to identify the call when **ConnectInboundCall** request is being executed.

Case 6. Inbound. Start interview by IVR, transfer to a live agent then.



Note that the 'inbound call id' here is not CATI call id. It's a dialer side id that allows dialer to identify the call when **ConnectInboundCall** request is being executed.

Case 7. Inbound. Start interview by live agent, transfer to IVR then.



Open Dialer Inbound and IVR Support API

[Dialer->CATI] `NotifyInboundCall(inboundCallId, inboundLineId, callerPhoneNumber)`

Notifies CATI about new connected inbound call.

[CATI->Dialer] `ConnectInboundCall(predictiveCampaignId, inboundCampaignId, inboundCallId)`

Should be used in predictive mode. Instructs dialer to choose an available agent from the predictive pool and connect the specified inbound call to the agent.

[CATI->Dialer] `ConnectInboundCallToAgent(inboundCampaignId, agentId, inboundCallId)`

Instructs dialer to connect the specified inbound call to the specified agent.

Q: *Should `ConnectInboundCall*` be synchronous operation or asynchronous (`NotifyOutcome` is expected in this case)?*

[CATI->Dialer] `DropInboundCall(inboundCallId)`

Instructs dialer to drop the specified inbound call. The method can be used if an inbound call can't be routed to agent for some reason. The method should not be used for routed calls (***CompleteCall*** and ***Hangup*** should be used instead).

[CATI->Dialer] `TransferToIvrAgent()`

Initiates transfer from a real Agent to an IVR virtual Agent. The both, predictive and non-predictive modes should be supported. It's still a question who should manage the virtual Agents – Dialer or CATI. Also, the question is should ***TransferToIvrAgent()*** command be synchronous or asynchronous (with no notification)?

[Dialer->CATI] `NotifyIvrAgentTransfer()`

Notifies CATI about IVR transfer result. The notification should contain at least the result (success/error) itself and other information like for instance virtual Agent ID.

[CATI->Dialer] `IvrRenderVoiceXml()`

Instruct dialer to play a prompt to respondent and capture some input if requested. All the instructions are sent to dialer as a VoiceXML statement. Currently we support a subset of VoiceXML as described in the 'VoiceXML Subset' section below.

[Dialer->CATI] `NotifyIvrSubmit()`

Notifies CATI about some IVR input related to preceding ***IvrRenderVoiceXml()*** command. The data is passed as a <key, value> collection.

[CATI->Dialer] `TransferToLiveAgent()`

Initiates transfer from IVR virtual Agent to a live Agent. The question is should ***TransferToLiveAgent()*** command be synchronous or asynchronous (with no notification)?

[Dialer->CATI] `NotifyLiveAgentTransfer()`

Notifies CATI about transfer to live Agent result. The notification should contain at least the result (success/error) and Agent ID.

Virtual IVR Agent Login

The Virtual Agents need to be logged in to dialer in the same way as Live Agents. There are at least 2 options how to implement this:

1. One-by-one login
2. Batch login

In order to simplify things at least for the first version most likely we should use the ***p.1*** approach. ***P.2*** is a sort of optimization that could be done later.

Another question is how to identify the Virtual Agent to dialer. Seems, possible options are:

1. Introduce a new attribute.
2. Introduce a new dialing mode.
3. Introduce a new binding type.
4. Use implicit identification by the 'station id'.

Virtual IVR Agents should login with 'no campaign' and can be shared between different campaigns.

Questions

1. Who should manage the virtual IVR Agents – Dialer or CATI?
2. Should CATI log in the IVR Agents to the dialer?
3. Should **TransferToIvr** and **TransferToAgent** commands be synchronous or asynchronous (with no notification)?
4. Should CATI send explicitly **GoReady** command after successful **TransferToIvr**?
5. Should the **ConnectInboundCall** be synchronous or asynchronous?

VoiceXML Subset

Limitations

1. Only 1 form per request
2. ECMAScript is not supported

Play a Synthesized Voice

```
<?xml version="1.0"?>
<vxml version="2.1">
  <form>
    <block>
      <prompt>
        Hello.
      </prompt>
    </block>
  </form>
</vxml>
```

Play an Audio File

```
<?xml version="1.0"?>
<vxml version="2.1">
  <form>
    <block>
      <prompt>
        <audio src="http://somehost.com/wav/hello.wav">
          Hello.
        </audio>
      </prompt>
    </block>
  </form>
</vxml>
```

The text included within the audio tag in the example above is something like the ALT text for images in HTML. If the platform is unable to open or play the source ("src") file in the audio tag, it falls back on generating TTS from the included text. The text may be omitted and the tag may be written using the short notation:

```
<audio src=" http://somehost.com/wav/hello.wav"/>
```

User Interaction with DTMF

Grammars are used by speech recognizers to determine what the recognizer should listen for, and so describe the utterances a user may say. Starting with VoiceXML Version 2.0, the W3C requires that all VoiceXML platforms must support

at least one common format, the XML Form of the W3C [Speech Recognition Grammar Specification \(SRGS\)](#). The only mode we currently support is “dtmf”.

```
<?xml version="1.0"?>
<vxml version="2.1">
<form id="mainmenu">
  <field name="menuchoice">
    <grammar type="application/srgs+xml" root="ROOT" mode="dtmf">
      <rule id="ROOT">
        <one-of>
          <item>1</item>
          <item>2</item>
          <item>3</item>
        </one-of>
      </rule>
    </grammar>
    <prompt>
      For sales, press 1.
      For tech support, press 2.
      For company directory, press 3.
    </prompt>
  </field>
</form>
</vxml>
```

Built-in Grammars

To simplify things some base grammars can be built into the dialer. They can be referenced by name in the “type” attribute of the “field” tag. We assume that dialer realizes at least “digits” grammar. A certain amount of digits can be specified with use of **length=n** parameter.

```
<?xml version="1.0"?>
<vxml version="2.1">
  <form>
    <field name="id" type="digits?length=7">
      <prompt>
        Enter your seven digit customer
        identification number.
      </prompt>
    </field>
  </form>
</vxml>
```

Wrong Input Handling

There should be possibility to trap and handle some exception conditions such as when the user enters no input or the user enters input not defined in the grammar for an input field. In the next example, we want to collect a seven digit identification number in a field.

```
<?xml version="1.0"?>
<vxml version="2.1">
  <form>
    <field name="id" type="digits?length=7">
      <prompt>
        Enter your customer identification number.
      </prompt>
      <noinput count="1">
        <prompt>
          Your identification number is the seven digit number on your membership card.
        </prompt>
        <reprompt/>
      </noinput>
      <noinput count="3">
        <prompt>
          It seems you are having difficulty with your identification number.
        </prompt>
      </noinput>
      <nomatch count="1">
        <prompt>
          Your identification number must be seven digits. Please try again.
        </prompt>
      </nomatch>
      <nomatch count="3">
        <prompt>
          It seems you are having difficulty with your identification number.
        </prompt>
      </nomatch>
    </field>
  </form>
</vxml>
```

In the above example, handlers are defined for the first two occurrences of the **nomatch** and **noinput** events, as well as separate handler for the third occurrence of each **nomatch** and **noinput** event.

The “count” attribute allows to avoid infinite loops and to increase the customer experience, for instance by helpfully transferring the user to customer service when they are having difficulty entering input.

Note that if the counter exceeds the maximum count defined, the highest handler should be executed. In the previous example, the system should continue to hit the count=”3” event once we have exceeded three **nomatch** or **noinput** events.

Sending Data to CATI

The **<submit>** tag should be treated by Dialer as instruction to send collected data (user input) to CATI via **NotifyIvrSubmit()** call. The data is passed as a <key, value> collection.

```
<?xml version="1.0"?>
<vxml version="2.1">

<form>
  <property name="termchar" value="#" />

  <field name="customerid" type="digits">
    <prompt>
      Please enter your customer identification number.
    </prompt>
  </field>

  <field name="age" type="digits?minlength=1;maxlength=2">
    <prompt>
      Please enter your age.
    </prompt>
  </field>

  <block>
    <prompt>
      Thank you.
    </prompt>
    <submit namelist="customerid age" />
  </block>
</form>

</vxml>
```

Audio Recording

The **<record>** tag is used to record audio from the user.

```
<?xml version="1.0"?>
<vxml version="2.1">
  <form>
    <record name="myrecording" beep="true">
      <prompt>
        Please record a message after the beep.
      </prompt>
    </record>
```

```
</form>
</vxml>
```

Variables

<var> tag is used to declare a variable. In order to keep the VoiceXML service stateless we need to pass some data to the dialer and get the data back with the **NotifyIvrSubmit()** response. Note, that according to the voiceXML, string values have to be enclosed in single quotes. Currently the dialer can return string values either enclosed in the quotes or not enclosed - it makes no difference, the value will be recognized in any case.

```
<?xml version="1.0" encoding="utf-8"?>
<vxml version="2.1" xml:lang="en-US" xmlns="http://www.w3.org/2001/vxml">
  <form>
    <property name="termchar" value="#" />
    <var name="__prevPageProgress" expr="'0.75;0.25'" />
    <var name="__pagemasterid" expr="'321'" />
    <var name="__theme" expr="'Main+theme'" />
    <var name="l" expr="'9'" />
    <var name="__sres" expr="'0|0'" />
    <var name="__sid__" expr="'8AQchCgSQdGLG6-OT18KG__uY7sueYQDqIZ4iGtFQYUjKGt6cm6-9Xaw_hUUTyV0i2t
mX_XbiNN9j1X-7lFEBys5H-48CiA8YpTBm5r4q4t9aFutSVOfy6BRkx_Zppj3w-6Rh_93xHWLDag9JQs3IAwMed3Fd_qgmHtch
z150ZX0P-3pngke0MZzw-WtUvjgmMUrqiidHa0rXqRmsslWwA2'" />
    <record name="q3" type="audio/x-wav" beep="true" maxtime="20s" finalsilence="10s" dtmfterm="true">
      <prompt>
        <audio src="http://localhost/isa/PYAMLMFAKKOYIBEYNHKRJRIBIMIEALXOH/p1000191/q3.wav">Please,
say something</audio>
      </prompt>
      <filled>
        <submit namelist="q3 __prevPageProgress __pagemasterid __theme l __sres __sid__" />
      </filled>
    </record>
  </form>
</vxml>
```

Generic DTMF Recognizer Properties

timeout

Default Value: 3 seconds

The time after which a **noinput** event is thrown by the platform. The “timeout” global property cannot be set higher than 60 seconds.

termchar

Default Value: “#”

The “termchar” global property allows the user to enter an optional terminating character after entering a DTMF input and before the termtimeout has elapsed.

```
<?xml version="1.0"?>
<vxml version="2.1">

<form>
  <property name="termchar" value="*"/>
  <field name="customerid" type="digits">
    <prompt>
      Please enter your customer identification number followed by the star key.
    </prompt>
  </field>

  <block>
    <submit namelist="customerid"/>
  </block>
</form>

</vxml>
```

termmaxdigits

Default Value: false

By setting “termmaxdigits” to true, one can allow for no timeout as soon as the user has matched the maximum number of digits.

Interdigittimeout

Default Value: 1 second

The interdigit timeout value to use when recognizing DTMF input.